

MEETILY

Meetily & Meetily Workflows

A beginner-friendly technical and contributor guide

Purpose

Explain what Meetily is, how Community and Pro differ, how the Automation/Agent API works, what the meetily-workflows repository is for, and the expected contribution lifecycle: announce work with an issue → build in your own repository → submit metadata to the catalog through a PR.

Scope: Meetily Community • Meetily Pro • Agent/Automation API • HTTP • CLI • MCP • Webhooks • Community workflow catalog

Start here

This guide follows a meeting from capture to automation, then explains how a contributor makes that automation discoverable through the community catalog.

If you are new to Meetily: read Sections 1–3 for the product overview and Community versus Pro comparison.

If you want to build an automation: continue through Sections 4–5, then follow the example in Section 9.

If you want to contribute: focus on Sections 6–8 and use the review checklist in Section 10.

Terms to know before you begin

- **API:** an interface through which a program requests data or actions from another program.
- **Webhook:** a notification sent to a configured destination when an event happens.
- **CLI:** a command-line tool used to issue commands from a terminal or script.
- **MCP:** a protocol that lets compatible AI assistants use tools exposed by a service.
- **Manifest:** a small metadata file describing a workflow and pointing to its source repository.
- **Pull request (PR):** a proposal to merge changes into a repository after review.

One example throughout: a workflow receives a notification that a meeting summary is ready, retrieves that summary, and posts it to a team chat. Its implementation lives in the contributor's own repository; the central catalog contains metadata that links to it.

Contents

1. Executive Summary	3
2. What Is Meetily?	3
3. Meetily Community vs Meetily Pro	5
4. Meetily Automation Architecture	6
5. Event-Driven Workflow: Subscribe → Verify → Deduplicate → Fetch → Act	7
6. What Is the meetily-workflows Repository?	8
7. Contributor Lifecycle: Issue → Build → Manifest → PR	9
8. Community Workflow Manifest	11
9. End-to-End Example	11
10. Engineering and Review Checklist	12
11. Terminology	13
12. Sources and Reference Links	14

1. Executive Summary

Meetily is a privacy-first AI meeting assistant. Its core job is to capture meetings, transcribe speech, and produce useful meeting summaries while keeping transcription and meeting data under the user's control. The open-source Community Edition focuses on local transcription and core meeting intelligence. Meetily Pro adds professional features and, importantly for workflow developers, an Automation/Agent API that lets external scripts, tools, and AI assistants interact with a running Meetily desktop app.

The meetily-workflows repository is not an execution engine and it is not the future Workflows product. It is an examples-and-catalog hub for automations that use capabilities already shipped in Meetily Pro: the local HTTP API, meetily-pro CLI, MCP server, and webhooks.

The key mental model

Meetily produces meeting intelligence. The Automation API exposes it safely. A contributor builds an automation in their own repository. The meetily-workflows repository stores a small manifest that describes and links to that external project so the community can discover it.

At a glance

Layer	What it does	Where it lives
Meetily Community	Local-first recording/transcription, AI summaries, imports and core meeting features.	Meetily desktop app / open-source repository
Meetily Pro	Professional features plus integrations for scripts and AI assistants.	Separate Pro product/codebase
Automation / Agent API	Local gateway used by HTTP, CLI, MCP and webhook integrations.	Running Meetily Pro instance
meetily-workflows	Examples + community catalog; does not execute contributor workflows.	Zackriya-Solutions/meetily-workflows
Contributor project	Actual automation/business logic.	Contributor's own GitHub repository

2. What Is Meetily?

Meetily is designed as a local-first AI meeting assistant. It can record meeting audio, transcribe it, and generate summaries. The supported Community architecture is a self-contained Tauri desktop application: a Next.js interface with a Rust/Tauri backend and native integration. The older Python/FastAPI backend in the repository is archived and is not the supported application path.

Core priorities

- Privacy and data sovereignty: transcription is designed to run locally rather than requiring cloud transcription.
- Useful meeting intelligence: convert raw meeting audio into searchable transcripts and structured summaries.

- Choice of AI: support local models and, where selected, external AI providers for summary generation.
- Cross-platform desktop use: Community supports Windows/macOS and can be built from source on Linux.
- Extensibility: Pro exposes the meeting system through HTTP, CLI, MCP and event-driven webhooks.
- Least-privilege integrations: API keys are scoped so automations receive only the permissions they require.

Typical user flow

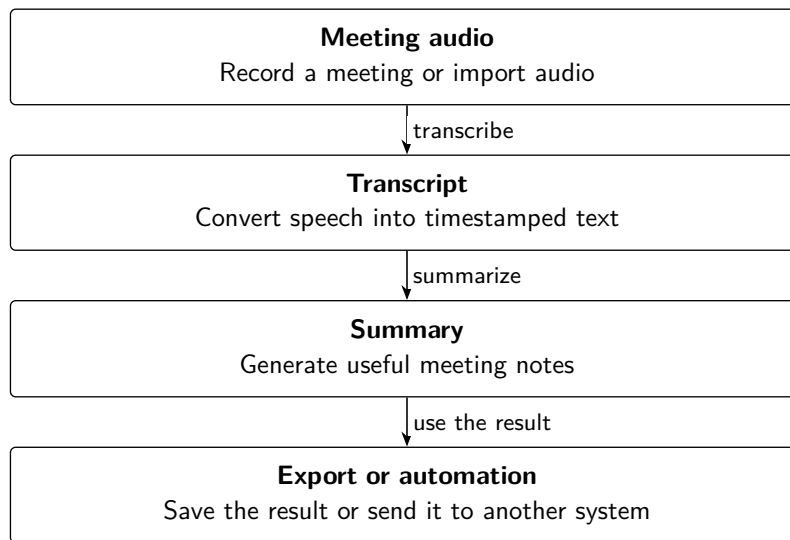


Figure 1. From meeting audio to a useful result.

The following table explains the same process step by step.

Step	User action	Meetily result
1	Start or import a meeting	Audio enters the local meeting workflow.
2	Transcribe	Speech is converted into timestamped transcript segments.
3	Review / identify speakers	Transcript can be refined; Pro includes speaker identification.
4	Summarize	A local model or configured AI provider produces a meeting summary.
5	Export or automate	Use exports directly, or let Pro integrations trigger downstream actions.

Privacy model

The important distinction is between transcription and optional summary providers. Meetily Pro states that transcription runs locally. If a user chooses BYOK (Bring Your Own Key) for summaries, transcript text can be sent to the selected AI provider; audio and recordings remain on the device. A fully local summary path is also available.

3. Meetily Community vs Meetily Pro

Community and Pro serve different needs. Community remains the free, MIT-licensed open-source edition. Pro is aimed at individuals and teams that need more advanced meeting workflows and integration capabilities.

Capability	Community Edition	Meetily Pro
License / positioning	Free, open source (MIT)	Commercial Pro product
Local transcription	Yes	Yes
AI summaries	Core summary support	Local AI or BYOK provider choices
Audio import / retranscription	Yes	Yes
Speaker identification	Not a core Community differentiator	On-device speaker diarization/identification
Custom summary templates	Limited/core experience	Yes
Advanced exports	Core exports vary by edition	PDF, DOCX and Markdown highlighted
Automation API	Not included yet in current docs	Yes
CLI / MCP / webhooks	Not included yet in current docs	Yes
Priority support	Community support model	Email priority support

Why Pro matters for this documentation

The current meetily-workflows examples target the shipped Meetily Pro Automation API. Official developer docs currently require Meetily Pro 1.11.0+ with an active license or trial for HTTP, CLI, or MCP integrations.

Meetily Pro integration surfaces

- HTTP API — direct local REST-style access from any language capable of HTTP requests.
- meetily-pro CLI — command-line client for scripts, terminals, and quick automation.
- MCP server — exposes Meetily to compatible AI assistants such as Claude Desktop/Code and Cursor.
- Webhooks — signed event notifications for event-driven workflows.
- SSE waits — bounded Server-Sent Event endpoints for waiting on job, recording, or summary terminal states.

4. Meetily Automation Architecture

The Agent API is a local HTTP gateway exposed by a running Meetily Pro desktop app. By default it is off. The documented loopback address is `http://127.0.0.1:8420`. The API follows Meetily's local-first posture: loopback access stays on the computer; LAN access and webhook destinations require explicit configuration/approval.

Conceptual architecture

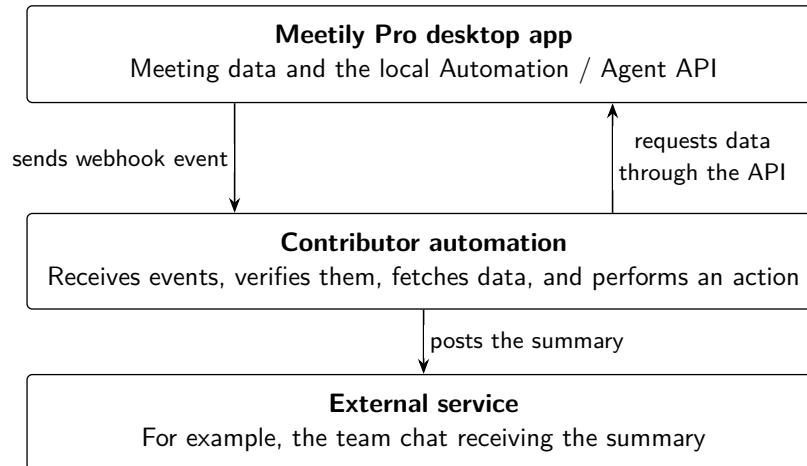


Figure 2. The automation connects Meetily to an external service.

The API returns the requested data to the automation after an authorized request. The contributor configures where the automation runs and how it reaches the Meetily instance. Storing the code in GitHub does not itself run the workflow. The central catalog does not execute it.

The integration surfaces are summarized below.

MEETILY PRO DESKTOP APP

↓ Local Agent / Automation API Gateway

HTTP API • meetily-pro CLI • MCP Server • Webhooks/SSE

↓

Your automation / AI assistant / external service

Security and permission model

Scope	Allows	Typical workflow need
read	Read meetings, transcripts, summaries, status, exports, webhooks.	Post a summary; archive meeting notes.
record	Start/stop/pause/resume recording; implies read.	Remote meeting-control workflow.
write	Rename meetings, edit speaker labels, set/regenerate summaries, control jobs/settings; implies read.	Post-process or regenerate summaries.

Scope	Allows	Typical workflow need
delete	Delete meetings; separate handling for stored media.	Cleanup workflow; use only when required.

A built-in loopback credential is read-only. Any automation that records, writes, or deletes must use a separately created key with the required scope. This is an important least-privilege design rule.

5. Event-Driven Workflow: **Subscribe** → **Verify** → **Deduplicate** → **Fetch** → **Act**

This five-stage pattern is the central workflow model used by the examples repository.

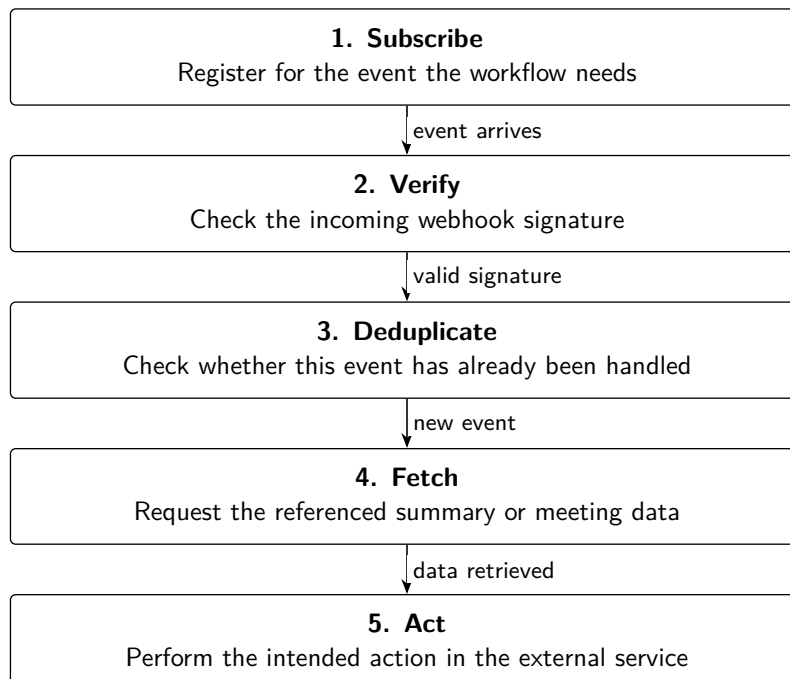


Figure 3. The successful event-handling path.

Reject an invalid signature. Handle a duplicate without repeating the external side effect. The blocks above show the successful path; errors require handling before the workflow can continue.

In the team-chat example, the notification says that the summary is ready. It does not contain the full summary: the automation fetches that content before posting it.

- 1. Subscribe:** register a webhook for the event(s) the automation needs.
- 2. Verify:** validate the signed webhook using the one-time HMAC secret and Meetily signature headers.
- 3. Deduplicate:** use `event_id` as the idempotency/deduplication key because at-least-once delivery can produce duplicates.
- 4. Fetch:** the webhook is intentionally thin; use `resource.id` and an authenticated API call to fetch the meeting, transcript, or summary.
- 5. Act:** perform the external action — for example create a task, post a summary, store notes, send a message, or update another system.

Important

Webhook payloads are notifications, not full meeting content. Transcript or summary text is not placed directly in the event envelope. The automation fetches the required content after receiving the event.

Common live triggers in the examples catalog

Catalog trigger	Backing event	Meaning
recording-ends	recording.stopped	Capture has stopped.
summary-ready	summary.completed	A summary operation completed.
import-finishes	job.completed (import)	An imported audio/job finished.
transcript-ready	transcription.completed	Dormant/reserved in the examples repo version; do not rely on it there.

6. What Is the meetily-workflows Repository?

The Zackriya-Solutions/meetily-workflows repository is an examples hub and a community catalog. It demonstrates how to build automations against the Meetily Agent API that already ships in Pro.

What it contains

- Runnable examples showing how to react to Meetily events and call the current API.
- A vendored `meetily_agent` Python helper for examples (explicitly not a supported SDK).
- Community workflow manifests under `community-workflows/`.
- Scripts that validate manifests and regenerate the catalog tables.
- Contribution guidance and conformance checks.

What it does not contain

- It is not the future Meetily Workflows execution product.
- It does not host or execute the full code of normal community workflows.
- It is not a connector framework or workflow engine.
- It should not be treated as an SDK package; the official surfaces are HTTP, CLI, MCP and the live OpenAPI manifest.

Separation of concerns

Contributor repository = implementation. meetily-workflows repository = discovery metadata + examples + validation.

7. Contributor Lifecycle: Issue → Build → Manifest → PR

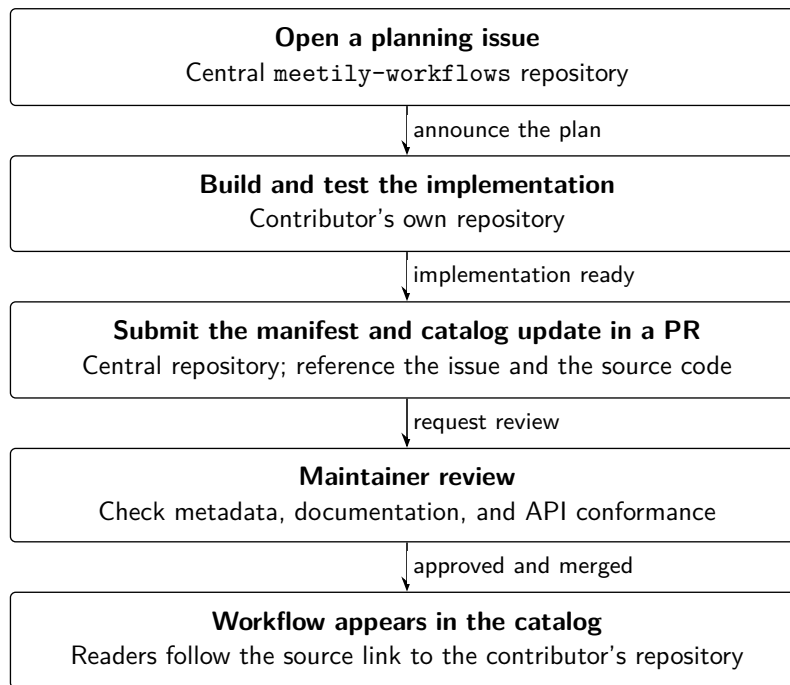


Figure 4. From a proposed workflow to a discoverable catalog entry.

Where each part belongs: the actual workflow code and setup instructions stay in the contributor's repository. The central repository receives the manifest and generated catalog changes. A catalog entry links to the implementation; publication does not deploy or run it.

How to read the guidance: the lifecycle below is the collaboration process described for this project. The issue and PR lists labeled “Recommended” are preparation guidance. The manifest schema and current validation checks determine which metadata is required; consult the linked contribution guide for the authoritative requirements.

For the collaboration process described for this project, contributors should make their intent visible before coding, keep the actual implementation in their own repository, and use the central repository to showcase the finished workflow.

1. Open an issue in meetily-workflows before starting. Describe the workflow being built, its purpose, intended trigger, external service/tool, expected scopes, and the contributor work-repository URL when available. This prevents duplicated effort and makes work discoverable.
2. Build the workflow in the contributor's own GitHub repository. The implementation should run standalone against the shipping Meetily surface rather than depending on unpublished Workflows-product internals.
3. Test against a real Meetily Pro instance. Confirm permissions, webhook approval behavior, HMAC verification, duplicate delivery handling, and the external action.
4. Add a catalog manifest under `community-workflows/<workflow-id>/manifest.yaml` in a branch/fork of meetily-workflows. The manifest points to the contributor's source repository; it does not copy the whole project into the catalog.

5. Run the catalog generator/validator and commit the generated catalog changes.
6. Open a pull request to meetily-workflows. Reference the original issue and include the meta-data/source repository so reviewers can inspect and, if needed, run the external implementation.
7. Maintainer review checks manifest validity, conformance with the shipped API, least-privilege scopes, standalone setup documentation, and real-world testing before the workflow is listed.

Recommended issue content

Field	What to provide
Workflow name	Clear human-readable name.
Problem / goal	What repetitive post-meeting task is being automated?
Trigger	recording-ends, summary-ready, import-finishes, manual, or scheduled.
Action	What happens after the trigger?
External tool	Slack, Notion, Jira, email, CRM, storage, custom service, etc.
Scopes	read / record / write / delete; request only what is required.
Implementation repo	Contributor-owned repository URL.
Status	Planned / in progress / ready for PR.

Recommended PR metadata

- References the planning issue (for example: Closes #<issue-number> or Related to #<issue-number>).
- Links the contributor's source repository and preferably a stable tag/commit.
- Explains trigger, scopes, language, entry point and run command.
- States the Meetily Pro version used for testing.
- Confirms webhook signature verification and idempotency where event-driven.
- Confirms setup instructions do not hard-code API secrets.
- Includes the generated catalog update and a manifest that passes validation.

8. Community Workflow Manifest

The manifest is the bridge between the central catalog and the contributor's external implementation. The folder name must match the manifest id.

```
manifest_version: 1
id: my-workflow
name: My Workflow
description: One line on what it does.
author: you (github.com/you)
language: python
trigger: summary-ready
scopes: [read]
source_url: https://github.com/you/your-repo
ref: v1.0.0
entry: run.py
run: "python run.py"
meetily_min_version: "1.11.0"
tags: [summary, integration]
license: MIT
```

Field interpretation

Field	Purpose
id	Unique slug; must match community-workflows/<id>/folder.
name / description	Human-facing catalog identity and short explanation.
author	Contributor identity/reference.
language	python, typescript, bash, or other.
trigger	How the workflow begins.
scopes	Exact Meetily permissions required.
source_url	External repository containing the real workflow code.
ref	Optional tag/commit pin for reproducibility.
entry / run	Entry file and exact standalone execution command.
meetily_min_version	Optional minimum compatible Meetily version.
tags / license	Discovery metadata and licensing information.

9. End-to-End Example

Example: a contributor wants to build “Summary to Team Chat,” which posts a completed Meetily meeting summary to a team channel.

1. Planning issue: contributor opens an issue explaining that the workflow reacts to summary-ready and posts the summary to a team-chat webhook. It needs Meetily read scope only.
2. Own repository: contributor creates `github.com/<user>/meetily-summary-to-chat` with README, setup, receiver, HMAC verification, deduplication storage and chat-posting logic.
3. Meetily setup: Automation API is enabled; CLI/read access is allowed; outgoing webhooks and the

destination are explicitly approved.

4. Runtime: Meetily emits `summary.completed`. The receiver verifies the signature, rejects/reuses duplicate `event_id` safely, fetches the summary by meeting id, and posts it to the external channel.
5. Catalog metadata: contributor adds `community-workflows/summary-to-chat/manifest.yaml` pointing `source_url` to the external repo.
6. PR: contributor opens a PR to `meetily-workflows`, references the original issue, includes testing information, and commits the regenerated catalog.

Data flow

Meeting finishes / summary generated

↓

Meetily Pro emits signed event notification

↓

Contributor workflow verifies + deduplicates + fetches summary

↓

External action (chat/task/CRM/storage/etc.)

10. Engineering and Review Checklist

- Use only capabilities that actually ship in the current Agent API / CLI / MCP / webhook surface.
- For event workflows, follow `subscribe` → `verify HMAC` → `deduplicate event_id` → `fetch` → `act`.
- Make the action idempotent and safe under duplicate or missed deliveries.
- Handle destinations that remain pending until a user explicitly approves them.
- Declare only the scopes the workflow genuinely needs.
- Keep tokens out of source code; use environment variables/token files or documented secure configuration.
- Document standalone setup and exact run command.
- If `record/write/delete` is required, document creation of an appropriately scoped key; the loopback credential is read-only.
- Test against a real Meetily instance.
- Validate `manifest.yaml` and regenerate the catalog before opening the PR.
- Remember that CI validates catalog metadata; it does not execute arbitrary contributor code.

11. Terminology

Term	Meaning
Meetily	Privacy-first AI meeting assistant.
Community Edition	Free/open-source Meetily edition.
Meetily Pro	Commercial edition with advanced features and current integration/automation surfaces.
Agent API / Automation API	Local API gateway exposed by a running Meetily Pro app.
Gateway	The local HTTP service through which API/CLI/MCP calls reach Meetily.
CLI	meetily-pro command-line client.
MCP	Model Context Protocol integration for compatible AI assistants.
Webhook	Signed HTTP event notification sent to an approved receiver.
SSE	Server-Sent Events used for bounded waits without polling.
Scope	Permission level attached to a token: read, record, write, delete.
Manifest	Small YAML metadata file that catalogs an external community workflow.
Workflows repo	Examples/catalog repository; not the workflow execution engine.

12. Sources and Reference Links

This document was compiled from the public Meetily repositories and official developer/product documentation available on 1 October 2026. The two share.google links supplied with the request did not expose readable report content through the available web interface, so they were not used as factual sources.

- **Meetily Community repository / README:** <https://github.com/Zackriya-Solutions/meetily>
- **Meetily Workflows repository:** <https://github.com/Zackriya-Solutions/meetily-workflows>
- **Meetily Workflows CONTRIBUTING.md:** <https://github.com/Zackriya-Solutions/meetily-workflows/blob/main/CONTRIBUTING.md>
- **Meetily developer documentation:** <https://docs.meetily.ai/developers>
- **Meetily API reference:** <https://docs.meetily.ai/developers/api-reference>
- **Authentication and scopes:** <https://docs.meetily.ai/developers/authentication>
- **Webhooks and SSE:** <https://docs.meetily.ai/developers/webhooks-and-sse>
- **meetily-pro CLI:** <https://docs.meetily.ai/developers/cli>
- **MCP server:** <https://docs.meetily.ai/developers/mcp>
- **Meetily Pro:** <https://meetily.ai/pro>

Source-of-truth rule

For Automation API behavior, the official developer documentation says the live OpenAPI 3.1 manifest at GET /openapi.json is canonical. If an examples repository or prose page disagrees with the running manifest, follow the live manifest.